부산대학교
PUSAN NATIONAL UNIVERSITY

Singly Linked Lists

# Giving Up Contiguity

Nodes, References, and the Head of a List

DS3600082-140
자료구조
2026학년도 2학기

**오용경**
데이터사이언스전공
yongkyung.oh@pusan.ac.kr

Section 1 • Singly Linked Lists

# Giving Up Contiguity

연속 저장 대신 참조로 순서를 표현한다

# 연속 저장의 장점과 비용

## 인덱스 접근은 빠르지만 가운데 삽입과 용량 확장에는 원소 이동이 필요하다

| 배열 연산 | 비용 | 추가로 필요한 작업 |
|---|---|---|
| 인덱스로 바로 접근 | $\Theta(1)$ | 없음 |
| 끝에 추가 | 상각 $\Theta(1)$ | 용량 부족 시 재할당 |
| $i$번째 위치에 삽입·삭제 | $\Theta(1 + n - i)$ | 뒤쪽 원소를 한 칸씩 이동 |

빠른 인덱스 접근과 원소 이동 비용은 모두 연속 저장에서 나온다

Source: Morin (2013), Chapters 2 and 3.

# 연결리스트

정의

## 연결리스트(linked list, 링크 리스트)

값 하나와 다음 노드를 가리키는 참조(reference)를 함께 담은 노드(node)를 이어 놓은 자료구조로, 노드가 메모리에서 이어져 있을 필요가 없다

Source: Morin (2013), Section 3.1.

# 노드가 놓인 위치와 목록의 순서는 다르다

## 메모리에서 떨어진 노드도 다음 노드(next)를 가리키는 참조를 따라 하나의 목록을 이룬다




연결리스트의 논리적 순서는 노드의 물리적 위치가 아니라 `next` 참조로 표현한다

Adapted from Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

# 같은 값 세 개, 다른 저장 방식

## 배열은 칸의 순서로, 연결리스트는 참조의 경로로 순서를 복원한다

| 표현 | 54 다음의 값을 알아내는 과정 |
|---|---|
| 배열 | 54가 있는 칸의 다음 인덱스를 읽어 26을 얻는다 |
| 연결리스트 | 54를 담은 노드의 `next`가 가리키는 노드에서 26을 읽는다 |

> 논리적 순서가 같아도 다음 값을 찾는 정보의 위치는 다르다

Adapted from Morin (2013), Chapters 2 and 3.

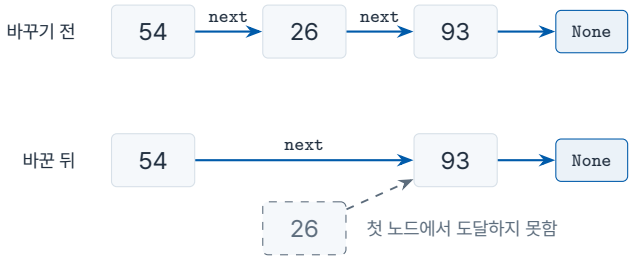
# 내부 표현이 바뀌면 같은 연산의 비용도 바뀐다

| | |
|---|---|
| **연산** | 조회·삽입·삭제 |
| **내부 표현** | 연속한 칸 또는 `next` 참조 |
| **내부 작업** | 주소 계산 또는 참조 추적 |
| **선택 기준** | 연산의 빈도와 비용 |

> 같은 목록 ADT라도 배열과 연결리스트는 연산을 수행하는 작업과 비용이 다르다

# 참조 한 개가 바뀌면 경로도 바뀐다

## 노드의 값은 그대로여도 `next`가 가리키는 대상에 따라 읽히는 순서가 달라진다




54를 담은 노드의 `next`가 26에서 93으로 바뀌면 경로는 54 → 93이 된다.

**Caution** 값을 저장한 노드가 존재하는 것과 첫 노드에서 그 노드에 도달하는 것은 다르다

Checkpoint

# Giving Up Contiguity

- 배열의 가운데 삽입에서 원소 이동이 필요한 이유는 무엇인가
- 연결리스트의 논리적 순서를 정하는 정보는 무엇인가
- 연속 저장을 포기하면서 잃는 장점은 무엇인가

Section 2 • Singly Linked Lists

# The Node

값과 다음 참조를 한 객체에 묶는다

# 노드 하나가 담는 것

마지막 노드 `u`




### 값과 연결

- `u.data`는 이 노드의 값이다.
- `u.next`는 다음 노드를 가리킨다.
- 마지막 노드의 `next`는 `None`이다.

Adapted from Morin (2013), Section 3.1.

# Python으로 노드를 만들면

## 노드는 값을 담는 data와 다음 노드를 가리키는 next를 갖는다

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

first = Node(54)
second = Node(26)
first.next = second

```

> first.next = second가 두 노드 사이의 순서를 만든다

Adapted from Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

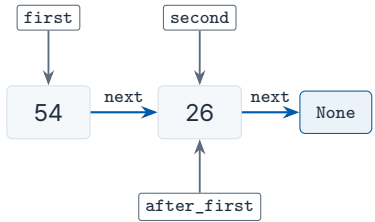
# next로 한 단계 뒤의 노드를 읽는다

## first.next는 second와 같은 노드를 가리킨다

```
first = Node(54)
second = Node(26)
first.next = second

after_first = first.next
print(after_first.data)          # 26
print(after_first is second)     # True
print(second.next is None)       # True

```




> next를 읽으면 값이 아니라 다음 노드를 가리키는 참조를 얻는다

# 두 번 따라가면 세 번째 노드에 이른다

## `.next`를 한 번 읽을 때마다 현재 노드에서 다음 노드로 한 단계 이동한다

```
third = Node(93)
second.next = third

print(first.data)              # 54
print(first.next.data)         # 26
print(first.next.next.data)    # 93
print(third.next is None)      # True

```

`first.next.next`는 세 번째 노드의 참조이고, `.data`를 읽어야 값 93을 얻는다

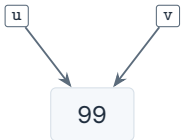
# 참조를 대입해도 노드는 복사되지 않는다

## 두 이름이 같은 노드를 가리키면 한쪽에서 바꾼 값이 다른 쪽에서도 보인다

```
u = Node(54)
v = u
v.data = 99

print(u.data)        # 99
print(u is v)        # True

```




이름 둘, 노드 하나

> `v = u`는 노드를 하나 더 만드는 문장이 아니라 같은 객체의 참조를 복사하는 문장이다

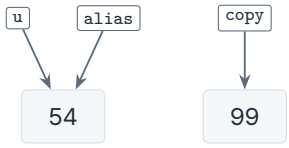
# 같은 노드를 가리키는 것과 새 노드를 만드는 것

## 두 노드의 값이 같더라도 같은 객체인지는 따로 확인해야 한다

```
u = Node(54)
alias = u
copy = Node(u.data)

print(alias is u)      # True
print(copy is u)       # False
copy.data = 99
print(u.data)          # 54

```




노드 둘

> 참조 대입은 한 객체의 이름을 늘리고, 생성자는 별도의 노드를 만든다

# 참조가 노드의 순서를 정한다

| 표현 | 다음 원소를 찾는 단서 | 순서를 정하는 것 |
|---|---|---|
| 배열 | 바로 옆 칸 | 메모리의 위치 |
| 연결리스트 | `next`가 가리키는 곳 | 노드에 저장한 참조 |

Source: Morin (2013), Section 3.1.

Checkpoint

# The Node

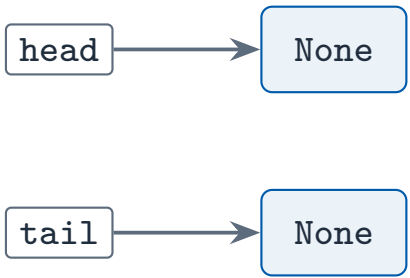
- 노드 하나가 함께 저장하는 두 정보는 무엇인가
- `first.next = second`가 표현하는 관계는 무엇인가
- `v = u` 뒤에 두 이름은 몇 개의 노드를 가리키는가

Section 3 • Singly Linked Lists

# Head and Tail

목록 객체와 노드 객체의 역할을 구분한다

# 빈 목록 객체의 head는 None을 가리킨다




목록 객체를 막 만들면 head와 tail이 모두 None이다. 두 참조는 첫 노드와 마지막 노드를 가리킨다.

Adapted from Morin (2013), Section 3.1.

# 목록 객체와 노드의 역할

| 담는 곳 | 저장하는 정보 | 역할 |
| --- | --- | --- |
| 목록 객체 | `head`, `tail`, `n` | 첫 노드·마지막 노드·원소 수 관리 |
| 각 노드 | `data`, `next` | 값과 다음 노드의 참조 저장 |

> 목록 객체는 첫 노드와 전체 상태를 관리하고, 각 노드는 값과 다음 노드의 참조를 저장한다

Adapted from Morin (2013), Section 3.1.

# 목록 객체의 시작 상태를 코드로 읽는다

## 노드가 없을 때 목록 객체는 머리, 꼬리(tail), 원소 수를 함께 기록한다

```
class LinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.n = 0

items = LinkedList()
print(items.head, items.tail, items.n)  # None None 0

```

> head와 tail은 노드의 값이 아니라 노드에 이르는 참조다

# head, tail, n을 함께 읽는다




$n = 3$이면 head에서 next를 두 번 따라가 tail에 이르고, tail.next는 None이다

Adapted from Morin (2013), Section 3.1.

# 노드가 하나일 때 지켜야 할 관계

**원소 수가 1이면 머리와 꼬리가 같은 노드를 가리키고 그 다음 노드는 없다**

| 확인할 식 | 의미 |
|---|---|
| `head is tail` | 두 참조가 같은 노드에 도달함 |
| `tail.next is None` | 더 뒤에 있는 노드가 없음 |
| `n == 1` | 목록에 속한 노드가 하나임 |

> 노드가 하나인 목록에서는 세 식이 모두 참이어야 하며, 하나라도 거짓이면 필드가 어긋난 것이다

# 목록 객체의 세 필드

| 이름 | 나타내는 것 | 없으면 |
|---|---|---|
| `head` | 첫 노드 | 첫 노드에 접근할 수 없음 |
| `tail` | 마지막 노드 | 끝에 붙이려면 $n-1$번 이동 |
| `n` | 담긴 원소 개수 | 셀 때마다 $\Theta(n)$ |

**Caution** `head`는 첫 노드를 가리키는 참조이지 첫 원소 자체가 아니다

Source: Morin (2013), Section 3.1.

# 노드의 존재와 목록 소속은 다르다

**목록에 속한 노드는 `head`에서 `next`를 따라 도달할 수 있어야 한다**

| 상태 | 첫 노드에서 보이는 경로 |
|---|---|
| `head`: 54 → 26 → 93 | 세 노드가 목록에 속하고 `n=3` |
| `head`: 54 → 93, 26은 별도 참조에만 남음 | 26 객체는 존재해도 이 목록에서는 보이지 않음 |

> `n`은 메모리에 존재하는 노드 수가 아니라 목록 경로에 속한 노드 수다

# 빈 목록의 `head`와 `tail`

## 노드가 하나도 없을 때의 상태를 정해 두어야 한다

| 상태 | `head`, `tail` | `n` |
|---|---|---|
| 빈 목록 | 모두 `None` | $0$ |
| 첫 원소 추가 뒤 | 같은 새 노드를 함께 가리킴 | $1$ |

> 연산마다 이 특별한 경우를 따로 확인해야 한다

Source: Morin (2013), Section 3.1.

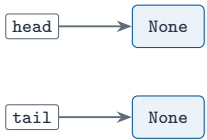
# head와 tail이 같은 노드를 가리킬 때

## 원소가 하나이면 두 참조가 같은 노드를 가리킨다

원소 하나, $n = 1$




마지막 원소 삭제 뒤, $n = 0$




**Caution** 한쪽만 바꾸면 다른 참조가 삭제한 노드를 계속 가리킨다

Source: Morin (2013), Section 3.1.

# 원소를 추가할 때 노드를 만든다

**배열은 용량을 미리 확보하고 연결리스트는 원소마다 노드를 할당한다**

| 시점 | 배열 | 연결리스트 |
|---|---|---|
| 만들 때 | 용량만큼 한 번에 | 노드 없음 |
| 원소 추가 | 빈 칸에 값을 씀 | 노드를 하나 만듦 |
| 원소 삭제 | 칸은 남음 | 목록의 연결에서 노드가 빠짐 |

> 연결리스트는 원소를 추가할 때 노드를 할당하므로 용량을 미리 정하지 않는다

Source: Morin (2013), Chapter 3.

Checkpoint

# Head and Tail

- 목록 객체와 노드 객체는 각각 무엇을 저장하는가
- 빈 목록의 `head`와 `tail` 값은 무엇인가
- 원소가 하나일 때 두 참조는 어떤 객체를 가리키는가

Section 4 • Singly Linked Lists

# References and C Pointers

Python 참조와 C 포인터의 공통점과 차이를 구분한다

# next가 다음 노드를 가리킨다

## C는 포인터(pointer), Python은 참조로 다음 노드와의 연결을 표현한다




Python: 참조, C: 포인터

> 두 언어 모두 다음 노드를 가리키지만 연결 정보를 다루는 연산은 다르다

Adapted from Morin (2013), Section 3.1.

# 두 선언을 나란히 읽으면

## Python과 C 모두 `next`로 다음 노드를 가리킨다

**PYTHON**

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

**C**

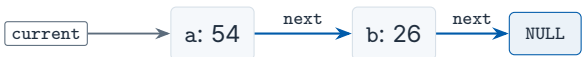
```c
struct Node {
    int data;
    struct Node *next;
};
```

> 두 코드에서 `next`는 다음 노드를 가리키지만 Python 참조에는 포인터 산술이 없다

# C에서도 다음 노드에 이르는 경로를 읽는다

**`current`가 `a`(값 54)를, `a.next`가 `b`(값 26)를 가리킨다고 가정한다**

| C 표현식 | 읽는 대상 |
|---|---|
| `&a` | 노드 `a`의 주소 |
| `current->next` | `a` 다음 노드 `b`의 주소 |
| `current->next->data` | `b`에 저장한 값 26 |




> Python의 `first.next.data`와 C의 `current->next->data`는 같은 연결을 따라 읽는다

# 두 언어에서 다르게 다루는 부분

| 구분 | C 포인터 | Python 참조 |
| --- | --- | --- |
| 빈 값 | `NULL` | `None` |
| 포인터 산술 | 가능 | 직접 제공하지 않음 |
| 메모리 회수 | 프로그래머가 관리 | 도달할 수 없는 객체를 실행 환경이 회수 |

**Caution** Python 참조를 숫자 주소처럼 더하거나 빼서 다른 객체로 옮길 수는 없다

Checkpoint

# References and C Pointers

- Python 참조와 C 포인터가 공통으로 맡는 역할은 무엇인가
- `None`과 `NULL`은 각각 무엇을 나타내는가
- C 포인터에서 가능한 연산 중 Python 참조로는 할 수 없는 것은 무엇인가

# What Do We Gain by Giving Up Contiguity?

**Next Lecture: Linked-List Operations**

- 순회로 원하는 자리까지 가는 방법
- 탐색, 삽입, 삭제의 연산별 비용
- `previous`와 `current`를 이용한 삭제


오용경 yongkyung.oh@pusan.ac.kr